Analysis of Algorithms 4

Iteratif

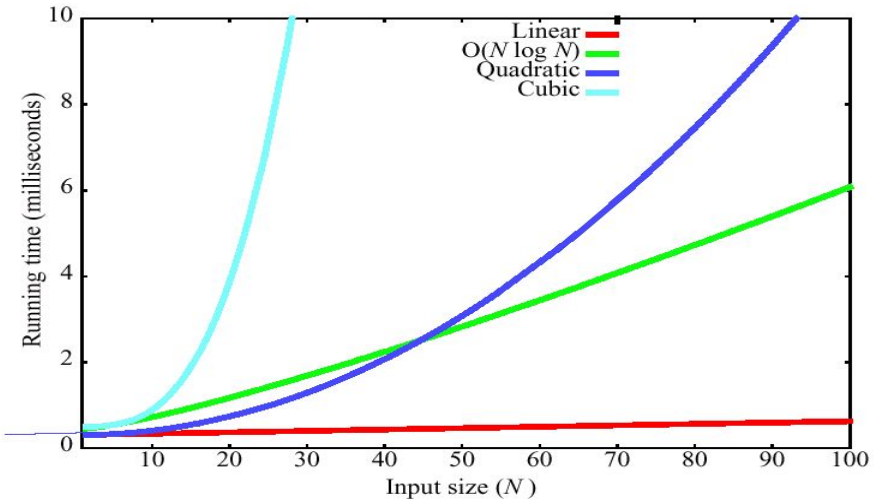
Slide disarikan dari berbagai sumber

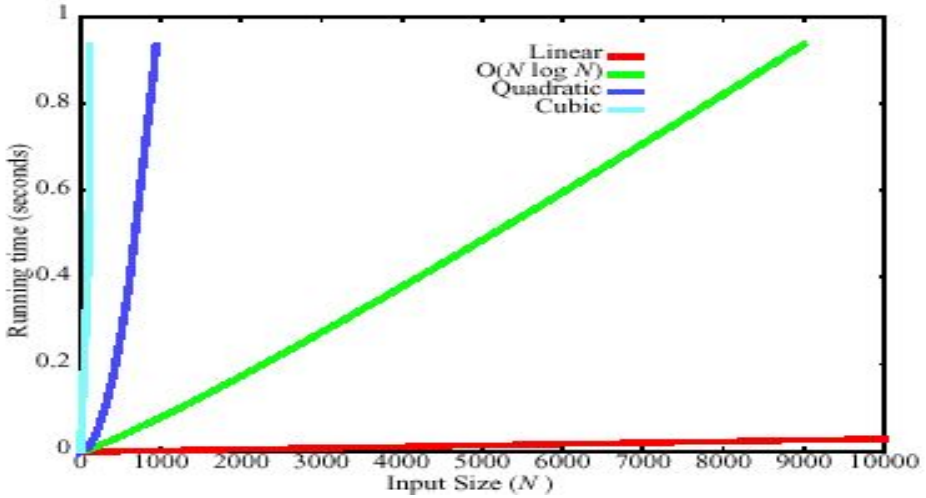
Tujuan

- Mahasiswa dapat menganalisis kompleksitas tipe algoritme iteratif
- Aproksimasi kompleksitas algoritme asimptotik berdasarkan struktur kontrol

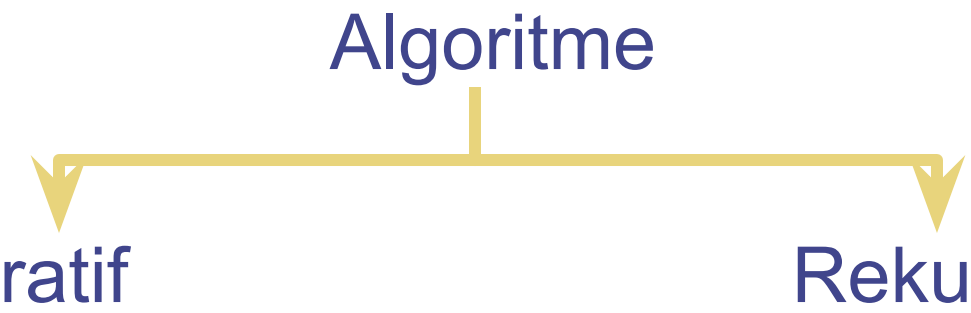
Running Time

- Untuk input yang sedikit, beberapa fungsi lebih cepat dibandingkan dengan yang lain
- Untuk input yang besar, beberapa fungsi memiliki running-time sangat lambat - tidak berguna





Kompleksitas Iteratif dan rekursif




```
A(n){
for  i := 1 to n do
  max(a,b)
}
```

```
A(n){
if(  )
  A(n/2)
}
```

- Iteratif: for loop, while loop, do while
- Rerkursif: calling function itself

Iterasi

- Dua jenis iterasi:
  - Count-Controlled: jumlah iterasi dapat ditentukan secara spesifik. Biasanya menggunakan counter. Misal: melakukan suatu operasi sebanyak n kali.
  - Condition-Controlled: iterasi terus-menerus dilakukan hingga suatu kondisi terpenuhi. Misal: membaca kemungkinan input sampai nilai -99 dibaca.
- Agar algoritme berhenti, iterasi harus memiliki kondisi berhenti dan memastikan kondisi berhenti tersebut tercapai. Jika tidak: infinite loop.

Iterasi

- Praktisnya, perhatikan bagian loop.
  - Jika tidak ada loop: f(n) = 1
  - Jika ada satu loop dari 1 ... n: f(n) = n
  - Jika ada nested loop bersarang, masing-masing 1 ... n: f(n) = n*n*...*n
  - Jika ada beberapa loop yang sekuensial: f(n) bergantung pada loop yang paling dominan.
- Bagaimana kalau bentuknya rekursif?
  - Subtitusi / pohon rekursi
  - Master Theorem

Kompleksitas konstan □ O(1)

- Waktu pelaksanaan algoritma adalah tetap, tidak bergantung pada ukuran masukan
- Tidak ada bagian loop pada program
- Contoh : printf(“hello“);

Kompleksitas kuadratik

```
for(i=0;i<n;i++)
    for (j=0; j;<n;j++)
        printf("hello")
```

- Berapa kali printf dieksekusi?
  - N*N kali

Kompleksitas kubik

```
for(i=0;i<n;i++)
   for (j=0; j;<n;j++)
      for(k=0;k<n;k++)
         printf("hello")
```

- Berapa kali printf dieksekusi?
  - N*N*N kali

O(NM)

- Inner loop dan outer loop memiliki batas akhir perulangan yang berbeda.

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        // code
    }
}
```

$O(LOG_C N)$

- Increment pada loop meningkat secara geometris (dikalikan dengan suatu konstanta tertentu).

```
for (int i = 1; i <=n; i *= c) {
    // some O(1) expressions
}
```

```
for (int i = n; i > 0; i /= c) {
    // some O(1) expressions
}
```

O(LOG LOG N)

- Increment pada loop meningkat secara eksponensial (dikalikan dengan suatu pemangkatan dari suatu konstanta tertentu).

```
1 for (int i = 2; i <=n; i = pow(i, c)) {
2     // some O(1) expressions
3 }
```

LOOP MAJEMUK

- Perhatikan loop yang paling dominan (lihat sifat big-Oh pada pertemuan sebelumnya).
- $\max(O(n), O(n^2), O(n)) = O(n^2)$

```
for (int i = 1; i <= n; i++) {
    // code
}
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        // code
    }
}
for (int i = 1; i <= n; i++) {
    // code
}
```

LATIHAN 1: BUBBLE SORT

```
1   procedure bubbleSort( list : array of items )
2       loop = list.count;
3
4       for i = 0 to loop-1 do:
5           for j = 0 to loop-1 do:
6               if list[j] > list[j+1] then
7                   swap( list[j], list[j+1] )
8                   swapped = true
9               end if
10          end for
11      end for
12  end procedure return list
```

LATIHAN 1: BUBBLE SORT

```
1   procedure bubbleSort( list : array of items )
2      loop = list.count;
3
4      for i = 0 to loop-1 do:
5         for j = 0 to loop-1 do:
6            if list[j] > list[j+1] then
7               swap( list[j], list[j+1] )
8               swapped = true
9            end if
10        end for
11     end for
12  end procedure return list
```

- Identifikasi input dan ukurannya terlebih dahulu.
- Tentukan 'pekerjaan' dominan yang ingin diperhitungkan.
- Perhatikan loop pada algoritme / program yang diberikan.

LATIHAN 1: BUBBLE SORT

- Ukuran input n = loop = list.count
- Pekerjaan = loop, pertukaran angka (swap)

```
1   procedure bubbleSort( list : array of items )
2      loop = list.count;
3
4      for i = 0 to loop-1 do:
5         for j = 0 to loop-1 do:
6            if list[j] > list[j+1] then
7               swap( list[j], list[j+1] )
8               swapped = true
9            end if
10        end for
11     end for
12  end procedure return list
```

O(n)

O(n)

O(1)

- Kompleksitas = $O(n)O(n)O(1) = O(n^2)$

LATIHAN 2: *PATTERN MATCHING*

Mencari indeks kemunculan string M dalam string N pertama kalinya.

Contoh:

N:THIS IS A TEST TEXT

M:TEST

Output: 10

Panjang string N >= string M.

Diberikan contoh program dalam bahasa C++ seperti berikut. Berapakah big-Oh dari algoritme tersebut.

```
1  void search(char* pat, char* txt)
2  {
3      int M = strlen(pat);
4      int N = strlen(txt);
5
6      for (int i = 0; i <= N - M; i++) {
7          int j;
8
9          for (j = 0; j < M; j++)
10             if (txt[i + j] != pat[j])
11                 break;
12
13         if (j == M)
14             cout << "Pattern found at index "
15                  << i << endl;
16     }
17 }
```

LATIHAN 2: *PATTERN MATCHING*

- Ukuran input m dan n yang merupakan ukuran kedua string
- Pekerjaan = jumlah perbandingan

```
1  void search(char* pat, char* txt)
2  {
3      int M = strlen(pat);
4      int N = strlen(txt);
5
6      for (int i = 0; i <= N - M; i++) {
7          int j;
8
9          for (j = 0; j < M; j++)
10             if (txt[i + j] != pat[j])
11                 break;
12
13         if (j == M)
14             cout << "Pattern found at index "
15                  << i << endl;
16     }
17 }
```

O(n-m)

O(m)

- Kompleksitas = O(n-m).O(m) = O(nm - m*m)

LATIHAN 3

```
1   scanf("%d", &n);
2   for(int i = 0; i < n*n; i++){
3       for(int j = 0; j < m; i*=2)
4           printf("A");
5       for(int j = 0; j < 2*m; i+=2)
6           printf("B");
7   }
8   for(int i = 0; i < n*m; i++)
9       printf("C");
10
```

LATIHAN 3

- Kompleksitas = $O(n^2)(\max(O(\log m), O(m)) + O(nm)$
  $= O(n^2).O(m) + O(nm) = O(n^2m)$

```
1   scanf("%d", &n);
2   for(int i = 0; i < n*n; i++){
3       for(int j = 0; j < m; i*=2)
4           printf("A");
5       for(int j = 0; j < 2*m; i+=2)
6           printf("B");
7   }
8   for(int i = 0; i < n*m; i++)
9       printf("C");
10
```

$O(n^2)$

$O(\log m)$

$O(m)$

$O(nm)$

LATIHAN 4: PERKALIAN MATRIKS

Misal input adalah matriks A berukuran n x m dan matriks B berukuran m x p.

- Input: matrices A and B
- Let C be a new matrix of the appropriate size
- For i from 1 to n:
  - For j from 1 to p:
    - Let $\text{sum} = 0$
    - For k from 1 to m:
      - Set $\text{sum} \leftarrow \text{sum} + A_{ik} \times B_{kj}$
    - Set $C_{ij} \leftarrow \text{sum}$
- Return C

LATIHAN 4: PERKALIAN MATRIKS

Ukuran input adalah matriks A berukuran n x m dan matriks B berukuran m x p.
Pekerjaan = perkalian.

- Input: matrices A and B
- Let C be a new matrix of the appropriate size
- For i from 1 to n: — O(n)
  - For j from 1 to p: — O(p)
    - Let $\text{sum} = 0$
    - For k from 1 to m: — O(m)
      - Set $\text{sum} \leftarrow \text{sum} + A_{ik} \times B_{kj}$
    - Set $C_{ij} \leftarrow \text{sum}$
- Return C

Kompleksitas = O(n).O(p).O(m) = O(npm)

LATIHAN 5: FIBONACCI ITERATIF

```
1   int fib(int n)
2   {
3     int f[n+2], i;
4
5     f[0] = 0;
6     f[1] = 1;
7
8     for (i = 2; i <= n; i++)
9         f[i] = f[i-1] + f[i-2];
10
11    return f[n];
12  }
```

Kompleksitas = O(n)

Kompleksitas Iteratif

```
func(n){
int i;
for  i = 1 to n do
  printf("analgor");
}
```

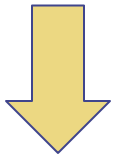
$O(n)$

```
func(n){
int i,j;
for  i = 1 to n do
  for  j = 1 to n do
     printf("analgor");
}
```

$O(n^2)$

Kompleksitas Iteratif

```
func(n){
int i;
for (i=1, i^2<=n, i++)
      printf("analgor");
}
```



$O(\sqrt{n})$

```
func(n){
int i;
for (i=1,i<=√n, i++)
      printf("analgor");
}
```

LATIHAN DESAIN ALGORITME 1

Rancanglah sebuah algoritme untuk menentukan jumlah faktor pembagi dari suatu bilangan bulat positif n.

Contoh Input:

10

Contoh Output:

4

Batasan:

$n <= 10^{12}$.

Kemampuan server: 10^{8} instruksi per detik.

Batas waktu: 1 detik

Apakah algoritme Anda dapat memenuhi batasan tersebut?

```
i:=0;
while i<n do
begin
   i:=i+1;
end;
```
1

```
i:= -2;
while i<n do
begin
   i:=i+1;
end;
```
2

```
i:=1;
while i<n do
begin
   i:=i*2;
end;
```
3

```
i:=0;
while i<n do
begin
   i:=i+1;
   j:=0;
   while j<n do
   begin
      j:=j+1;
   end;
end;
```
4

```
for(int i=0; i<n; i++)
{ a++; }
```
5

```
for(int i=0; i<n; i++)
{ a++; b++; }
```
6

```
for(int j=0; j<m; j++)
{
    for(int i=0; i<n; i++)
    {
        print ("a");
    }
}
```
7

```
for(int j=1; j<=n; j++)
{
    for(int i=1; i<=n; i*=10)
    {
        print ("a");
    }
}
```
8

```
for(int j=0; j<n; j++)
{
    for(int i=0; i<=j; i++)
    {
        print ("a");
    }
}
```
9

Latihan

Kompleksitas Iteratif

```
func(n){
int i=0, s=0;
  while(s<=n)
  {
     i++;
     s=s+i;
     printf("analgor");
  }
}
```

```
func(n){
int i, j,k,n;
for(i=1, i<= n, i++)
  {
  for(j=1, j<=i, j++)
     {
      for(m=1, m<=100, m++)
        {
        printf("analgor");
        }
     }
  }
}
```

Kompleksitas Iteratif

```
func(n){
int i, j,k,n;
for(i=1, i<= n, i++)
    {
    for(j=1, j<=i^2, j++)
        {
         for(m=1, m<=n/2, m++)
            {
            printf("analgor");
            }
        }
     }
}
```

```
func(n){
int i;
for(i=1, i<= n, i=i*2)
  printf("analgor");
}
```

Asumsi n>=2

```
func(n)
{
   while (n>1){
  printf("analgor");
  n=n/2;
   }
}
```

Kompleksitas Iteratif

```
func(n){
int i,j;
for(i=1, i<= n, i=++)
   for(j=1, j<=n, j=j+i)
        printf("analgor");
}
```

```
func(n){
int i, j,k,n;
for(i=n/2, i<= n, i++)
  {
  for(j=1, j<=n/2, j++)
     {
      for(m=1, m<=n, m=m*2)
        {
        printf("analgor");
        }
     }
  }
}
```

Kompleksitas Iteratif

```
func(n){
int i, j,k,n;
for(i=n/2, i<= n, i++)
  {
  for(j=1, j<=n, j=j*2)
     {
      for(m=1, m<=n, m=m*2)
        {
        printf("analgor");
        }
     }
   }
}
```

Jawaban

LATIHAN DESAIN ALGORITME 1

Jika Anda mengecek seluruh kemungkinan faktor dari 1 hingga n, kompleksitasnya adalah O(n).

```
1   int countDivisors(int n)
2   {
3       int cnt = 0;
4       for (int i = 1; i <= n; i++) {
5           if (n % i == 0) {
6               cnt++;
7           }
8       }
9       return cnt;
10  }
```

Waktu = $10^{12}/10^{8} = 10^{4}$ detik. *Time limit exceeded*.

LATIHAN DESAIN ALGORITME 1

Jika Anda mengecek seluruh kemungkinan faktor dari 1 hingga $\sqrt{n}$, kompleksitasnya adalah $O(\sqrt{n})$.

```
1   int countDivisors(int n)
2   {
3       int cnt = 0;
4       for (int i = 1; i <= sqrt(n); i++) {
5           if (n % i == 0) {
6               if (n / i == i)
7                   cnt++;
8               else
9                   cnt = cnt + 2;
10          }
11      }
12      return cnt;
13  }
14
```

Waktu = $10^6/10^8$ = 0.01 detik. *ACCEPTED!*

Kompleksitas Iteratif

```
i:=0;
while i<n do
begin
   i:=i+1;
end;
```

O(n)

```
i:= -2;
while i<n do
begin
   i:=i+1;
end;
```

O(n+2) ~ O(n)

```
i:=1;
while i<n do
begin
   i:=i*2;
end;
```

O(log n)

```
i:=0;
while i<n do
begin
   i:=i+1;
   j:=0;
   while j<n do
   begin
      j:=j+1;
   end;
end;
```

$O(n^2)$

Kompleksitas Iteratif

```
for(int i=0; i<n; i++)
{ a++; }
```

O(n)

```
for(int i=0; i<n; i++)
{ a++; b++; }
```

O(2n) ~ O(n)

```
for(int j=0; j<m; j++)
{
    for(int i=0; i<n; i++)
    {
        print ("a");
    }
}
```

O(mn)

```
for(int j=1; j<=n; j++)
{
    for(int i=1; i<=n; i*=10)
    {
        print ("a");
    }
}
```

O(n log n)

Kompleksitas Iteratif

```
for(int j=0; j<n; j++)
{
    for(int i=0; i<=j; i++)
    {
        print ("a");
    }
}
```

$\sum i = n(n+1)/2 \approx \frac{1}{2} n^2$

$O(n^2)$

Kompleksitas Iteratif

```
func(n){
int i=0, s=0;
  while(s<=n)
  {
     i++;
     s=s+i;
     printf("analgor");
  }
}
```

Loop will stop when s > n

Cari hubungan antara jumlah operasi yang dilakukan dengan n

i	1	2	3	4	5	6			k
s	1	3	6	10	15	21			n

$$\frac{k(k+1)}{2} \sim n \Rightarrow \frac{k^2+k}{2} \sim n \Rightarrow k \sim O(\sqrt{n})$$

Kompleksitas Iteratif

```
func(n){
int i, j,k,n;
for(i=1, i<= n, i++)
  {
  for(j=1, j<=i, j++)
      {
       for(m=1, m<=100, m++)
          {
          printf("analgor");
          }
      }
   }
}
```

Loop will stop when i > n

Cari hubungan antara jumlah operasi yang dilakukan dengan n

i	1	2	3	4	5	6			n
j	1 x	2 x	3 x	4 x	5 x	6 x			n x
m	1 x100	2 x100	3 x100	4 x100	5 x100	6 x100			n x100

$$k = 100 + 2 * 100 + 3 * 100 + 4 * 100 + 5 * 100 + 6 * 100 \ldots + n * 100$$

$$k = 100\,(1 + 2 + 3 + 4 + 5 + 6 \ldots + n)$$

$\sum i = n(n+1)/2 \approx \frac{1}{2} n^2$

$$k = 100\,(\frac{n(n+1)}{2}) \Rightarrow k = 100\,(\frac{n^2+n}{2}) \sim O(n^2)$$

Kompleksitas Iteratif

```
func(n){
int i, j,k,n;
for(i=1, i<= n, i++)
    {
    for(j=1, j<=i^2, j++)
        {
         for(m=1, m<=n/2, m++)
            {
            printf("analgor");
            }
        }
     }
}
```

Loop will stop when i > n

Cari hubungan antara jumlah operasi yang dilakukan dengan n

i	1	2	3	4	5	6			n
j	1 x	4 x	9 x	16 x	25 x	36 x			n^2 x
m	1 x $\frac{n}{2}$	4 x $\frac{n}{2}$	9 x $\frac{n}{2}$	16 x $\frac{n}{2}$	25 x $\frac{n}{2}$	36 x $\frac{n}{2}$			n^2 x $\frac{n}{2}$

$$k = \frac{n}{2}(1 + 4 + 9 + 16 + 25 + 36 \ldots + n^2)$$

$$\sum i^2 = n(n+1)(2n+1)/6 \approx 1/3\ n^3$$

$$k = \frac{n}{2}(\frac{n(n+1)(2n+1)}{6}) \quad \Rightarrow \quad O(n^4)$$

Kompleksitas Iteratif

```
func(n){
int i;
for(i=1, i<= n, i=i*2)
  printf("analgor");
}
```

Loop will stop when i > n

Cari hubungan antara jumlah operasi yang dilakukan dengan n

k	1	2	3	4	5	6			k
i	2	4	8	16	32	64			n

$n \sim 2^k$  $k \sim O(\log_2 n)$

```
func(n){
int i;
for(i=1, i<= n, i=i*3)
  printf("analgor");
}
```

$k \sim O(\log_3 n)$

Kompleksitas Iteratif

Loop will stop when n=1

Cari hubungan antara jumlah operasi yang dilakukan dengan n

```
Asumsi n>=2

func(n)
{
  while (n>1){
  printf("analgor");
  n=n/2;
   }
}
```

n	2	4	8	16	32				n
k	1	2	3	4	5				k

$2^k \sim n$

$k \sim O(\log_2 n)$

Kompleksitas Iteratif

```
func(n){
int i,j;
for(i=1, i<= n, i=++)
   for(j=1, j<=n, j=j+i)
        printf("analgor");
}
```

Loop will stop when i>n

Cari hubungan antara jumlah operasi yang dilakukan dengan n

i	1	2	3	4		n
j	1 to n	1 to n	1 to n	1 to n		1 to n
k	n	$\frac{n}{2}$	$\frac{n}{3}$	$\frac{n}{4}$		$\frac{n}{n}$

$$k = n + \frac{n}{2} + \frac{n}{3} + \frac{n}{4} + \cdots + \frac{n}{n} \Rightarrow k = n(1 + \underbrace{\frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \cdots + \frac{1}{n}}_{logn})$$

$$O(n(logn))$$

Kompleksitas Iteratif

```
func(n){
int i, j,k,n;
for(i=n/2, i<= n, i++)
  {
  for(j=1, j<=n/2, j++)
     {
      for(m=1, m<=n, m=m*2)
        {
        printf("analgor");
        }
     }
   }
}
```

$\frac{n}{2}$ times

$\frac{n}{2}$ times

$\log_2 n$ times

Independent loop

$$\frac{n}{2} * \frac{n}{2} * \log_2 n \quad \sim \quad O(n^2 \log_2 n)$$

Kompleksitas Iteratif

```
func(n){
int i, j,k,n;
for(i=n/2, i<= n, i++)
  {
  for(j=1, j<=n, j=j*2)
     {
      for(m=1, m<=n, m=m*2)
        {
        printf("analgor");
        }
     }
  }
}
```

$\frac{n}{2}$ times

$\log_2 n$ times

$\log_2 n$ times

Independent loop

$$\frac{n}{2} * (\log_2 n)^2 \sim O(n(\log_2 n)^2)$$

SELESAI

References : https://www.youtube.com/watch?v=FEnwM-iDb2g